

Universidad Tecnológica de Manzanillo
Ingeniería en Desarrollo y Gestión de Software
Materia: Desarrollo para dispositivos inteligentes

Alumnos:

Emmanuel Valencia Vázquez

Ulises Alejandro Curiel Pérez

Docente: Daniel German Fausto Solis

Grado: 9 Grupo: IDG 1

MediaNest TV

Fecha de entrega: 18 de julio de 2026

Tabla de contenido

1. Objetivo del proyecto
2. Problemática
3. Justificación
4. Alcance y productos entregables
5. Cotización estimada
6. Diagrama de Gantt
7. Roles y contribuciones
8. Arquitectura y datos
9. Tecnologías
10. Flujos de funcionamiento
11. Módulos
12. Pruebas
13. Seguridad, despliegue y evidencias
14. Conclusiones

1. Objetivo del proyecto

1.1 Objetivo general

Desarrollar MediaNest TV, una aplicación web nativa para televisores LG con webOS que permita descubrir, consultar y reproducir películas abiertas, series de prueba y música desde una interfaz optimizada para control remoto, conservando de forma local los favoritos y el progreso de reproducción.

1.2 Objetivos específicos

- Implementar una interfaz de 1280 × 720 con foco visible y navegación mediante flechas, OK/Enter y BACK.
- Organizar el contenido en Inicio, Películas, Series, Música, Buscar y Mi espacio.
- Consultar catálogos y resultados desde servicios públicos compatibles, y reproducir medios abiertos en la TV.
- Mostrar detalle, episodios y controles de reproducción; permitir avanzar, retroceder, pausar y salir.
- Guardar favoritos, recientes y progreso en localStorage para conservarlos en el navegador o TV.
- Empaquetar la aplicación como archivo .ipk para pruebas con webOS.

2. Problemática que se busca resolver

Las pantallas inteligentes suelen usarse para consumir contenido, pero muchas soluciones requieren suscripciones, cuentas o catálogos cerrados. Para una práctica académica se necesitaba una aplicación funcional que demostrara navegación de televisión, consumo de APIs y reproducción multimedia sin depender de un backend propio ni de un servicio de pago.

MediaNest concentra contenido audiovisual abierto y resultados públicos en una sola experiencia de TV. El reto técnico consiste en que cada acción pueda realizarse a distancia, con poco texto, controles grandes y una respuesta clara del foco, además de manejar resultados de distintos proveedores y conservar el estado útil del usuario localmente.

3. Importancia y justificación

El sistema permite aplicar desarrollo web en un entorno de pantalla inteligente, donde el teclado y el cursor no son el medio principal. Integra HTML, CSS, JavaScript compatible con webOS, reproducción de audio y video, persistencia local y consumo de servicios web. Como prototipo, demuestra que una interfaz ligera puede funcionar en hardware de TV y puede empaquetarse para su instalación de prueba.

4. Alcance del sistema y productos entregables

Producto	Alcance funcional	Tecnologías
Aplicación MediaNest TV	Navegación por secciones, tarjetas, detalle, búsqueda, reproducción y favoritos.	HTML, CSS, JavaScript ES5
Catálogo externo	Películas abiertas, series, música y búsquedas a servicios públicos.	Archive.org, TVMaze, iTunes, Audius, Jamendo y YouTube
Persistencia local	Favoritos, recientes y avance de reproducción en el dispositivo.	localStorage
Paquete webOS	Aplicación instalable para pruebas en TV LG.	appinfo.json, webOS CLI, .ipk

No se incluye registro de alumnos, captura de calificaciones, historial académico, administración de usuarios, login ni base de datos remota, porque no corresponden al propósito de una aplicación de entretenimiento para TV. Por el mismo motivo, los roles administrador, docente y alumno no aplican en la versión actual; existe un único usuario local de la pantalla.

4.1 Usuarios y escenarios de uso

El usuario objetivo es la persona que opera la televisión mediante control remoto. No se solicita correo, contraseña ni información personal. Puede explorar el catálogo, abrir un detalle, seleccionar un episodio, ejecutar una búsqueda y reproducir un medio. La sección Mi espacio permite conservar elementos de interés en ese mismo dispositivo. Si el proyecto evoluciona a un servicio multiusuario, se podrán introducir los perfiles administrador, editor de catálogo y espectador; en la versión entregada estos perfiles no existen y por tanto no se simulan.

4.2 Requisitos funcionales

- RF-01: el sistema debe responder a flechas, OK/Enter, Escape y código BACK de LG.
- RF-02: el sistema debe presentar tarjetas de contenido y permitir abrir una vista de detalle.
- RF-03: el sistema debe buscar películas, series y música en proveedores externos disponibles.
- RF-04: el reproductor debe pausar, reanudar, retroceder 10 segundos, avanzar 30 segundos y cerrarse.
- RF-05: el usuario debe poder guardar y retirar favoritos, así como reanudar contenido reciente.
- RF-06: la app debe poder empaquetarse con la configuración requerida por webOS.

5. Cotización estimada

La siguiente estimación corresponde a un prototipo académico en pesos mexicanos (MXN); no contempla publicación comercial, licencias de contenido, infraestructura de streaming ni soporte posterior.

Concepto	Esfuerzo	Tarifa/h	Subtotal
Análisis, requisitos y diseño	18 h	\$250	\$4,500
Interfaz y navegación para TV	34 h	\$300	\$10,200

Concepto	Esfuerzo	Tarifa/h	Subtotal
Catálogo, búsqueda y reproducción	36 h	\$300	\$10,800
Persistencia y configuración webOS	16 h	\$280	\$4,480
Pruebas, empaquetado y documentación	22 h	\$250	\$5,500

Resumen	Importe
Subtotal de desarrollo	\$35,480 MXN
Contingencia técnica (10 %)	\$3,548 MXN
Total estimado	\$39,028 MXN

6. Diagrama de Gantt (estimado vs. real)

Cronograma estimado y real de MediaNest



Figura 1. Cronograma estimado y real del desarrollo de MediaNest TV.

7. Roles, organización y contribuciones

Integrante	Rol	Funciones específicas
Emmanuel Valencia Vázquez	Líder técnico y desarrollador frontend/webOS	Diseño de arquitectura, estructura HTML, estilos CSS, lógica JavaScript, integración de APIs, reproductor, localStorage, configuración de empaquetado y corrección de incidencias.

Integrante	Rol	Funciones específicas
Ulises Alejandro Curiel Pérez	Calidad, pruebas y apoyo funcional	Diseño y ejecución de pruebas, validación de navegación con control remoto, revisión de usabilidad, registro de incidencias, verificación de contenido y apoyo en evidencias y documentación.

7.1 Aportación detallada

Emmanuel desarrolló los módulos de navegación, catálogo, detalle, búsqueda, reproducción y persistencia. Utilizó HTML, CSS, JavaScript ES5, APIs REST mediante fetch y herramientas de webOS. Los problemas principales fueron la compatibilidad de navegación en TV y la variación de resultados externos; se resolvieron con un sistema central de foco, mensajes de estado, filtros y fuentes alternativas de contenido.

Ulises realizó pruebas exploratorias y funcionales sobre el flujo completo: cambio de secciones, apertura de tarjetas, búsqueda, guardado, reproducción, pausas, avance/retroceso y regreso. Documentó resultados y reportó comportamientos de interfaz para que fueran ajustados. Las evidencias se encuentran en el código organizado del proyecto, el paquete .ipk generado y las capturas incorporadas en esta documentación.

8. Arquitectura del sistema y persistencia

8.1 Diagrama de arquitectura

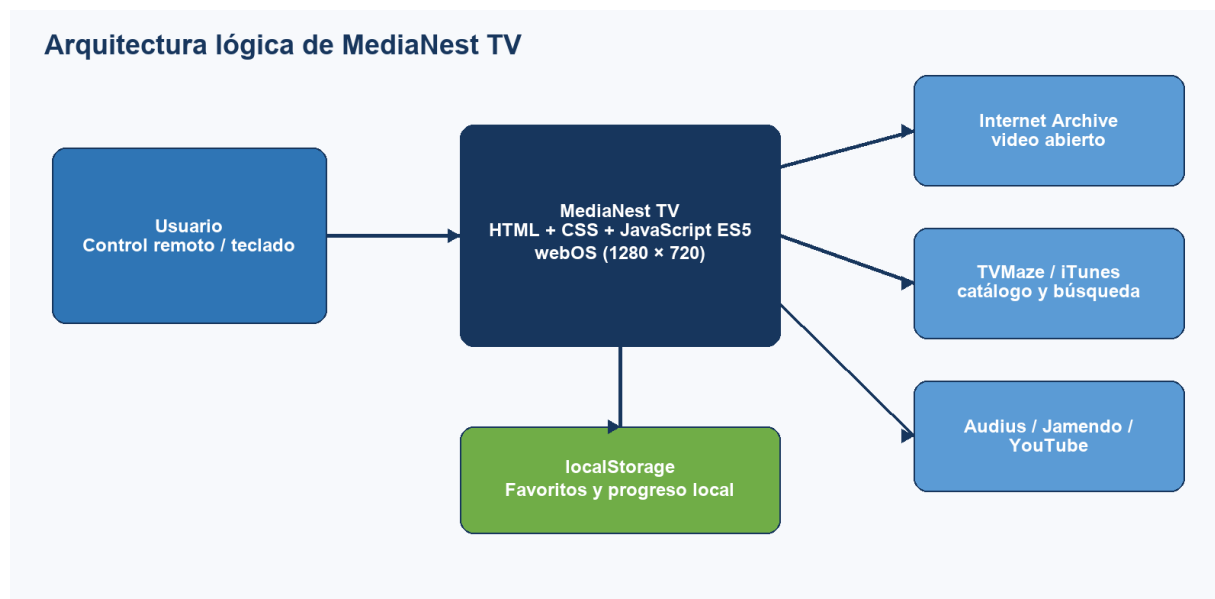


Figura 2. Arquitectura lógica de MediaNest TV.

La aplicación se ejecuta en la TV o navegador y consulta servicios públicos por HTTPS. No existe un backend propio ni una base de datos remota. La única persistencia actual es localStorage: medianest.saved.v2 para favoritos y medianest.recent.v2 para recientes y progreso.

8.2 Modelo de datos

Entidad lógica	Campos principales	Relación
Contenido	id, title, kind, section, image, description, source, mediaType	Un contenido puede tener episodios; puede guardarse o aparecer en recientes.
Episodio	id, title, durationText, description, image, source	Pertenece a una serie de prueba.
Favorito local	id y copia de datos del contenido	Lista local del usuario de la TV.
Reciente local	id, progreso y datos del contenido	Lista local; se actualiza al reproducir.

No hay diagrama ER relacional porque MediaNest no utiliza tablas SQL ni colecciones de una base de datos. El modelo anterior describe las estructuras JSON que se reciben o se guardan en el navegador. Esto evita afirmar una base de datos inexistente y delimita correctamente el alcance técnico.

9. Tecnologías utilizadas

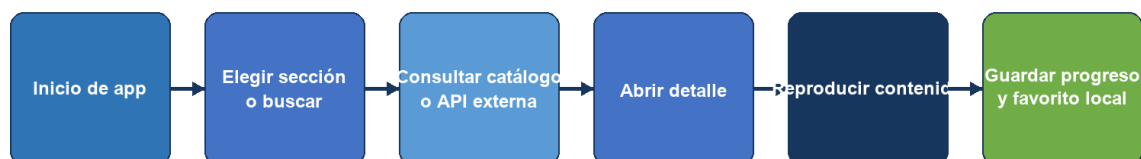
Capa	Tecnologías y librerías	Uso
Frontend	HTML5, CSS3 y JavaScript ES5	Interfaz, estilos y lógica compatible con navegadores webOS.
Multimedia	video, audio e iframe HTML5	Reproducción de videos, audios y contenido musical embebido.
Servicios externos	Archive.org, TVMaze, iTunes, Audius, Jamendo y YouTube Data API	Consulta de catálogos, búsqueda y fuentes de reproducción.
Plataforma	LG webOS, appinfo.json, @webos-tools/cli	Empaquetado e instalación de prueba como .ipk.
Persistencia	Web Storage API (localStorage)	Conservación local de favoritos y progreso.

La capa frontend se separa en index.html para la estructura, styles/main.css para presentación y scripts/app.js para comportamiento. scripts/data.js ofrece contenido inicial para la demostración, mientras que scripts/config.js reúne valores opcionales de integración. Esta organización facilita localizar cada responsabilidad sin requerir un framework de gran tamaño.

10. Diagramas de funcionamiento

10.1 Flujo de navegación y reproducción

Flujo de operación principal



BACK cierra reproductor o detalle; las flechas cambian el foco y OK/Enter confirma la acción.

Figura 3. Flujo principal de MediaNest TV.

11. Descripción de módulos

Módulo	Descripción
Inicio y navegación	Muestra contenido recomendado, acceso a secciones y foco controlado con teclado o control remoto.
Películas y series	Presenta contenido abierto, detalles, metadatos y episodios cuando la entrada corresponde a una serie.
Música y búsqueda	Consulta proveedores de audio y video; permite filtrar por tipo y ordenar los resultados visibles.
Reproductor	Abre una capa de reproducción con pausa, avance, retroceso, salida y barra de progreso.
Mi espacio	Guarda favoritos, recientes y avance del contenido en el almacenamiento local.
Configuración	Centraliza claves opcionales y URL del reproductor externo en scripts/config.js.

12. Plan de pruebas y resultados

Prueba	Resultado esperado	Resultado
Navegación con flechas y Enter	El foco cambia y se activa el elemento seleccionado.	Correcto en el flujo de interfaz.
Cambio de sección	Se actualizan título, filtros y catálogo.	Correcto.
Búsqueda de contenido	Se consultan resultados y se muestran tarjetas.	Correcto; depende de la disponibilidad de APIs externas.

Prueba	Resultado esperado	Resultado
Reproducción	El medio se abre y responde a pausa, avance, retroceso y BACK.	Correcto para fuentes compatibles.
Favoritos y progreso	El estado permanece al recargar en el mismo dispositivo.	Correcto mediante localStorage.
Empaquetado webOS	Se genera un archivo .ipk instalable para pruebas.	Correcto: com.emman.medianest_0.1.0_all.ipk.

Los errores potenciales detectados se relacionan principalmente con servicios externos: contenido no disponible, límites de API, políticas de reproducción o problemas de red. La aplicación informa estados y conserva un catálogo de prueba para evitar una interfaz vacía. Como mejora, se recomienda agregar una capa propia de servidor para normalizar fuentes y almacenar métricas sin exponer configuración sensible en el cliente.

12.1 Criterios de aceptación

Se considera aceptada la versión cuando el usuario puede recorrer todas las secciones sin perder el foco, abrir contenido de prueba, cerrar las capas con BACK, guardar un favorito, recargar la página y encontrarlo nuevamente en Mi espacio. Las pruebas de conectividad se ejecutan por separado, porque la disponibilidad final depende de Internet y de las políticas de los servicios de terceros.

13. Seguridad, rendimiento, usabilidad, despliegue y evidencias

13.1 Requisitos no funcionales

- Seguridad: no se manejan cuentas ni contraseñas. La configuración con claves de terceros debe restringirse o trasladarse a un backend antes de una publicación real.
- Rendimiento: se utiliza JavaScript sin frameworks pesados y una interfaz fija de 1280 × 720 para reducir carga en hardware de TV.
- Usabilidad: botones grandes, foco visible, navegación por flechas, OK/Enter y BACK; los textos describen cada acción.
- Disponibilidad: el catálogo de prueba permite una demostración inicial; las funciones en línea dependen de la conexión y disponibilidad de cada proveedor.

13.2 Configuración y despliegue

El entorno requiere Node.js y dependencias instaladas con npm install. La aplicación se ejecuta localmente con npm run dev y se abre en http://localhost:8080. Para LG webOS se utiliza la CLI @webos-tools/cli; npm run package:webos genera el .ipk y los comandos ares permiten instalar y lanzar la aplicación en una TV configurada en Developer Mode. La entrega no requiere hosting, dominio ni servidor propio porque es una app instalable; su código fuente se publica en GitHub para control de versiones y consulta académica.

13.3 Evidencias de entrega

- Código fuente: carpeta app con index.html, styles/main.css, scripts/app.js, scripts/data.js y scripts/config.js.
- Configuración webOS: app/appinfo.json y package.json en la raíz.
- Paquete instalable: com.emman.medianest_0.1.0_all.ipk.
- Documento general: DocumentoGeneral_MediaNest.docx y su versión PDF.
- Repositorio del proyecto: <https://github.com/Sage1985/medianest>.
- No se entregan scripts de base de datos porque el sistema no usa una base de datos. La clave de YouTube no se publica en el código; cada instalación debe configurar su propia credencial si requiere esa integración.

14. Conclusiones, logros y mejoras futuras

MediaNest TV cumple el objetivo de demostrar una aplicación para pantalla inteligente con navegación remota, catálogo multimedia, búsqueda, reproducción y persistencia local. El desarrollo permitió aplicar compatibilidad con webOS, consumo de APIs y diseño centrado en televisión. El aprendizaje principal fue adaptar una interfaz web al uso con control remoto, donde el foco y la continuidad del flujo son determinantes.

Como mejoras futuras se propone implementar autenticación opcional, sincronización de favoritos entre dispositivos, un backend que proteja las claves y establezca las fuentes, control parental, telemetría, caché de catálogo, un panel de administración y publicación mediante repositorio y hosting. Estas ampliaciones deberán acompañarse de una política de privacidad y pruebas en el modelo exacto de TV objetivo.